

Introduction to Linked Lists

Node Structure & Dynamic Memory Allocation

Unit II • Lecture 15



Recap: the two things arrays cannot do well

Problem 1: fixed size

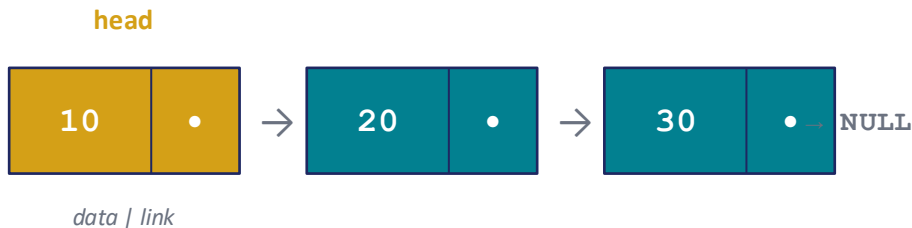
`int arr[100];` commits to 100 slots whether you use them or not. Too small and you're stuck; too big and you waste memory.

Problem 2: costly insert/delete

Inserting in the middle means SHIFTING every element after it — $O(n)$ work just to make room.

- You already saw the array's answer to problem 1 fail: Lecture 11's linear queue ran out of room on the right while the left sat empty.
- A linked list solves BOTH problems at once: it grows one node at a time, and inserting is just rewiring two pointers — no shifting.

What is a linked list?



- A sequence of NODES, each storing data plus the address of the next node.
- The whole list is reachable from ONE pointer, called head.
- The last node's link is NULL — the signpost that says "stop here".
- Nodes are NOT next to each other in memory — only the arrows make it a sequence.

Definition to note: a linked list is a linear collection of nodes, where each node points to the next, and the list is accessed through a single head pointer.

The node: one struct, two jobs

```
struct Node
{
    int data;
    struct Node *next;
};
```

Exactly what you built before

Same shape as the linked stack's Node (L9) and the linked queue's Node (L14). This struct is the one idea underneath every linked structure in the course.

- data holds the actual value — here a plain int, but it could be any type.
- next holds the ADDRESS of the following node, or NULL if this is the last one.
- struct Node *next is legal INSIDE the struct's own definition because a pointer only needs to know an address's size, not the whole struct's size.

Dynamic memory allocation: the concept

Static (arrays)

```
int arr[100];
```

Size fixed at COMPILE time. Memory reserved the moment the program starts, whether used or not.

Dynamic (linked list)

```
malloc(...)
```

Memory requested at RUN time, exactly when a node is needed. The program decides its own memory size while running.

- Dynamic memory comes from a region called the HEAP — separate from the stack where ordinary local variables live.
- Three functions matter: malloc (request memory), free (return memory), and the NULL check (did the request succeed?).

malloc() up close

```
struct Node *head;

head = malloc(sizeof(struct Node));

if (head == NULL)
{
    printf("Allocation failed\n");
    return 1;
}
```

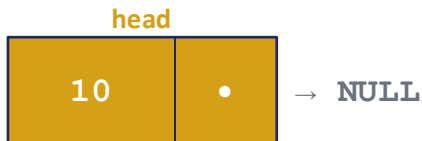
- `sizeof(struct Node)` tells `malloc` exactly how many bytes one node needs.
- `malloc` returns the ADDRESS of that memory — store it in a pointer.
- `malloc` can fail (heap exhausted) and returns `NULL` — always check before using the pointer.

`#include <stdlib.h>` is required — that is where `malloc` and `free` are declared.

Building the list by hand — step 1: one node

```
struct Node *head = NULL;  
  
head = malloc(sizeof(struct Node));  
head->data = 10;  
head->next = NULL;
```

- head starts as NULL — no list yet, exactly like top = NULL or front = NULL before.
- One malloc creates one node.
- next = NULL because right now this node is BOTH the first and the last.



Step 2: add a second node by hand

```
struct Node *second =  
    malloc(sizeof(struct Node));  
second->data = 20;  
second->next = NULL;  
  
head->next = second;    /* link them */
```

- second is built exactly like head was — malloc, fill data, next = NULL.
- The ONE new line: head->next = second. This is what turns two separate nodes into one connected list.
- Notice head itself never changes — it still points at the node holding 10.



Step 3: a third node completes the list

```
struct Node *third =  
    malloc(sizeof(struct Node));  
third->data = 30;  
third->next = NULL;  
  
second->next = third;    /* link them */
```

- Same pattern a third time: malloc, fill, link the PREVIOUS node to this one.
- A 3-node list needed exactly 3 mallocs and 2 linking lines — the pattern scales by repetition, which is exactly what a loop will do in Lecture 16.



Why head is the only pointer you must never lose

The whole list lives behind one address

second and third are just local variables — useful for building the list by hand, but not required to KEEP it. Once head->next->next reaches node 3, second and third can disappear and the list is still complete and reachable.

- head->data is 10. head->next->data is 20. head->next->next->data is 30.
- If head is ever overwritten or lost, EVERY node becomes unreachable memory — not deleted, just lost (a memory leak).
- This is exactly the same responsibility top carried for the stack, and front carried for the queue.

Traversal: the loop that replaces hand-naming

```
for (struct Node *p = head;  
    p != NULL;  
    p = p->next)  
{  
    printf("%d -> ", p->data);  
}  
printf("NULL\n");
```

- p is a temporary walking pointer — head itself never moves.
- Start at head, stop when NULL is reached, step forward with p = p->next each time.
- Exactly the loop pattern from stackDisplay (L9) and qDisplay (L14) — same idea, no stack or queue rules attached.

Output for our 3-node list: 10 -> 20 -> 30 -> NULL — verified by compiling and running this exact code.

Array vs Linked List — the comparison that started today

Aspect	Array	Linked List
Size	Fixed at compile time	Grows one node at a time
Memory layout	Contiguous	Scattered, joined by pointers
Access element i	$\text{arr}[i] — O(1)$	Walk from head — $O(n)$
Insert / delete middle	Shift elements — $O(n)$	Rewire two pointers — $O(1)$ once found
Extra memory per element	None	One pointer (next)

Neither wins outright — each is faster at exactly what the other is slow at. Lecture 16 builds this properly with a loop.

Nodes, links, and one pointer called head

- A linked list is a chain of nodes (data + next), reachable through a single head pointer, ending in NULL.
- malloc requests heap memory at run time; always check for NULL before using the result.
- Linking is one line: `previousNode->next = newNode` — building a list is just repeating this pattern.
- Losing head loses the entire list (a memory leak) — it is the one pointer that must never be overwritten carelessly.
- Traversal walks with a temporary pointer, head untouched — the pattern every future list operation builds on.

EXIT TICKET By hand, write the malloc + linking code for a 4th node holding 40, added after the node holding 30.